STARE: Packing Spanning Tree for GPU-GPU Communication

Bill Qian
Yale College
New Haven, Connecticut, USA

Micah Gold
Yale College
New Haven, Connecticut, USA

Alex Schapiro
Yale College
New Haven, Connecticut, USA

Abstract

Distributed training performance in modern artificial intelligence is heavily dominated by collective communication operations such as AllReduce, Broadcast, and ReduceScatter. State-of-the-art frameworks rely on high-performance collective libraries, most notably NVIDIA's NCCL, which typically utilize ring-based or hierarchical algorithms optimized for maximum bandwidth efficiency. While highly effective in homogeneous environments, contemporary research demonstrates that alternative network topologies can significantly reduce latency and improve scalability under heterogeneous network conditions. Microsoft Blink, a research prototype, proposed a spanning-tree-based approach (packing spanning trees) to implement these collectives, showing robust performance on fragmented hardware allocations. However, Blink's codebase remains proprietary.

This paper details our effort to reconstruct, implement, and open-source Blink-style spanning tree collectives directly into the NCCL stack. We document our system architecture, including graph extraction, Multiplicative Weight Update (MWU) algorithms for tree packing, and greedy refinement heuristics. We describe our success in, for the first time, integrating a packed-spanning tree approach to AllReduce into NCCL (for both intra-node and inter-node cases) and show throughput gains over standard NCCL.

Keywords

NCCL, GPU Communication, Collective Communication, Packed Spanning Trees, Network Topology, Distributed Deep Learning, Multi-Node Routing

1 Introduction and Motivation

The training of Large Language Models (LLMs) and advanced deep neural networks requires massive parallelization across distributed GPU clusters. The bottleneck in distributed training is fundamentally shifting from raw compute to efficient GPU-GPU collective communication. State-of-the-art frameworks rely on the NVIDIA Collective Communication Library (NCCL) to execute operations like *AllReduce*, *AllGather*, *ReduceScatter*, and *Broadcast*.

Currently, NCCL maps these operations onto logical communication topologies, predominantly leveraging Ring and Binary Tree algorithms. At startup, NCCL selects the logical communication topology that best fits the underlying physical network topology. While ring-based algorithms (requiring $O(p)$ steps for p nodes) are optimal for perfectly homogeneous and fully connected systems, physical network topologies in real-world clusters are increasingly complex, fragmented, and heterogeneous. A vast majority of the systems used in production are not purpose-built for fast inter-GPU communication, and hence use a variety of interconnect types, including PCIe, Ethernet (inter-node), NVLink, NVSwitch, etc. For example, a distributed training cluster hosted on AWS or on Yale HPC might use a heterogeneous mixture of A100s, H200s, Network Interface Cards (NICs), PCIe generations, and switch architectures. While the standard NCCL algorithms perform well on 'nice'-looking, homogeneous topologies, there are still optimizations to be made for heterogeneous architectures.

Our work is primarily inspired by the Blink paper [5], which proposes framing collective communication as a network flow problem, and solves it using dynamically generated, packed spanning trees. Blink demonstrated throughput gains of up to 8x compared to an old version of NCCL (2.6) released four years ago. While the research is public, the implementation is not.

The primary goal of this project is to reconstruct, implement, and open-source Blink-style collectives, integrating them directly into NCCL, and report the performance of Blink on large clusters. By undertaking this project, we hope to bridge the gap between academic research and deployable AI infrastructure.

2 Background

2.1 NCCL as an Algorithm-Selection System

NCCL (and its derivatives) is the standard collective communication library used by many distributed GPU training systems. It implements communication primitives such as AllReduce, AllGather, ReduceScatter, Broadcast, and Reduce over PCIe, NVLink, NVSwitch, InfiniBand, and socket-based inter-node networks [4]. NCCL does not implement these collectives using a single fixed algorithm. Instead, it builds topology-aware communication graphs and selects among multiple algorithm families, protocols, and channel configurations based on the detected hardware topology, message size, number of ranks, and available hardware features.

Current NCCL includes several algorithm families, including Ring, Tree, CollNetChain, CollNetDirect, NVLS, NVLSTree, and PAT [4]. These algorithms cover different points in the latency-bandwidth-design space. Ring is bandwidth efficient for large homogeneous transfers; Tree reduces communication depth; CollNet and NVLS use hardware or network support when available; and PAT improves AllGather and ReduceScatter behavior for latency-sensitive regimes. STARE adds a different scheduling primitive to this set:

Blink-style packed spanning trees generated from the topology graph. The goal is to integrate this schedule generation into NCCL's existing graph-construction and algorithm-selection path rather than replacing NCCL's runtime or framework-facing API.

2.2 Ring

The Ring algorithm arranges the participating ranks into one or more logical cycles. For AllReduce, the standard ring schedule consists of a ReduceScatter phase followed by an AllGather phase. During ReduceScatter, each rank sends one chunk to its successor, receives one chunk from its predecessor, and accumulates partial reductions. After $p-1$ steps for p ranks, each rank owns one reduced shard. During AllGather, these reduced shards circulate for another $p-1$ steps until every rank has the full reduced tensor.

The main advantage of Ring is bandwidth efficiency on homogeneous topologies. Each rank sends and receives approximately $2(p-1)S/p$ bytes for a tensor of size S, which approaches $2S$ bytes per rank as p grows. This makes Ring effective for large messages when all logical ring edges have similar bandwidth. Its main limitation is the linear startup cost: the algorithm requires $O(p)$ communication steps. It is also sensitive to bottlenecks in the logical cycle, since a slow or congested edge can limit the progress of the entire ring.

2.3 Tree and Double Binary Tree

Tree algorithms organize ranks into a rooted communication tree. A reduction moves data from the leaves toward the root, and a broadcast moves data from the root back toward the leaves. For AllReduce, these two phases can be composed as reduce followed by broadcast. A balanced tree reduces the number of communication steps from $O(p)$ to $O(\log p)$, which improves latency for small and medium-sized messages. The cost is that a single tree may not use all available links evenly, especially on dense or heterogeneous topologies.

Modern NCCL uses a double-binary-tree construction for AllReduce rather than a single tree [2]. The tensor is split across two complementary binary trees, so that ranks that act as internal forwarding nodes in one tree are leaves in the other. This balances forwarding work across ranks and recovers much of the bandwidth efficiency of Ring while preserving logarithmic communication depth. However, the construction is still a fixed tree family. It does not solve the general problem of constructing many topology-specific trees whose combined edge usage matches an arbitrary capacitated graph.

2.4 Hardware-Assisted NCCL Algorithms

NCCL also includes algorithms that use hierarchy or hardware support when it is available. CollNet-style algorithms use a hierarchical structure in which GPUs first communicate within a node, then selected representatives participate in a network-level collective, and the result is distributed back within each node. CollNetChain uses local chains within a node, while CollNetDirect uses a more direct local communication pattern. These algorithms are useful when the network or communication stack can support collective operations more efficiently than a pure software ring or tree.

NVLS and NVLSTree target systems with NVLink SHARP and NVSwitch support. NVLS uses hardware-assisted collective execution inside the NVSwitch domain, allowing reduction or multicast-like behavior to be handled by the fabric. NVLSTree combines this local NVLS behavior with a tree-like structure at a higher level, such as across nodes or across larger communication domains. These algorithms can provide high performance on supported systems, but they are specialized to specific hardware features and hierarchy assumptions. They do not provide a general method for generating optimal schedules on arbitrary heterogeneous topologies.

2.5 PAT

PAT, or Parallel Aggregated Trees, is an NCCL algorithm for AllGather and ReduceScatter introduced to improve latency-sensitive communication regimes [3]. It is based on Bruck-style and binomial-tree communication ideas. Instead of using a linear ring schedule, PAT executes multiple shifted tree schedules in parallel. This reduces the number of network transfer steps for small and medium messages and works for arbitrary rank counts.

PAT is relevant to STARE because it shows that NCCL continues to incorporate new algorithm families when a fixed schedule improves an important collective regime. However, PAT remains a predefined algorithmic family for particular collectives. It does not synthesize a schedule from a capacitated topology graph. STARE targets this remaining gap by adding a topology-generated packed-tree schedule to NCCL's graph-construction path.

2.6 Blink and Packed Spanning Trees

Blink models the communication topology as a directed graph $G = (V, E)$ with capacities c_e on edges [5]. It generates collective schedules by packing multiple directed spanning trees over this graph. The spanning trees are algorithmically chosen such that they maximize the overall throughput. While Blink is similar to NCCL in dividing a tensor into chunks to be sent across multiple channels, the advantage of Blink is that these trees are weighted to fit the shape of the topology. This allows the schedule to use multiple paths through the topology and reduces dependence on any single bottleneck link.

Blink generates candidate trees using a multiplicative-weight update procedure. Initially, each edge has a cost related to its capacity. In each iteration, Blink finds a low-cost directed spanning tree and then increases the weights of the edges used by that tree. This encourages later trees to avoid already-used bottleneck edges and explore other parts of the graph. The resulting packed trees are especially useful on fragmented or heterogeneous allocations, where fixed NCCL schedules may fail to exploit available path diversity. STARE

follows Blink's basic packed-tree approach, but integrates the mechanism directly into NCCL.

2.7 ForestColl and Theoretical Optimality

ForestColl is the most relevant theoretical comparison for STARE [6]. Like Blink, it represents collective schedules using broadcast and aggregation trees. Unlike Blink's MWU-based heuristic, ForestColl characterizes the maximum achievable throughput of a capacitated topology for allgather. ForestColl provides an optimality benchmark for STARE rather than a replacement for its systems contribution. STARE addresses the problem of incorporating Blink-style packed spanning trees into NCCL's production graph-construction path. This differs from computing an optimal schedule in isolation, because NCCL imposes additional constraints from its channel representation, transport layer, CUDA execution model, fallback behavior, and algorithm selector.

3 Algorithm Design and Implementation

We designed our implementation to serve as another transport option, rather than the default transport layer for all workloads. Therefore, our implementation replaces the tree-generation phase in NCCL while leaving ring and NVLS paths untouched. We use an environment variable `NCCL_BLINK=1` to gate the implementation. Enabling and disabling STARE requires a recompilation. The execution pipeline is divided into graph extraction, tree generation, and scheduling.

3.1 Graph Extraction

The first algorithmic step is to translate NCCL's hardware topology into a capacity graph. In our graph, nodes represent the actual hardware devices and an edge represents the communication route between two devices along with a capacity that approximates the bandwidth along that route. STARE's motivation is strongest on irregular hardware, where some GPU pairs are connected by high-bandwidth links, others by lower-bandwidth routes, and some paths may be absent. A ring collective is constrained by the weakest edge on its logical cycle, whereas a set of spanning trees can distribute traffic across multiple routes and reduce dependence on any single bottleneck.

The extractor walks NCCL's full intra-host topology and emits every node type that contributes to GPU-to-GPU bandwidth: the GPUs themselves, the NVSwitch fabric (modeled as a single Steiner vertex with one edge per attached GPU), the PCIe root ports and their downstream bridges, the CPU sockets, the host NICs, and the per-NIC NET endpoints. Edges carry a typed link class (NVLink, PCIe, inter-CPU SYS, NET) and a per-link capacity. The H200 base topology produces a 37-vertex 86-edge graph: 8 GPU + 1 NVS + 8 PCI + 2 CPU + 9 NIC + 9 NET vertices, with 16 NVL edges, 50 PCI edges, 2 SYS edges, and 18 NET edges. Figure 1 shows the same extractor parsing a 10-host A5000 cluster, with each host's intra-node links (PCIe/SYS/NET) plus the synthetic non-blocking Fabric vertex that carries inter-host NIC capacity. Including PCIe and CPU vertices in the

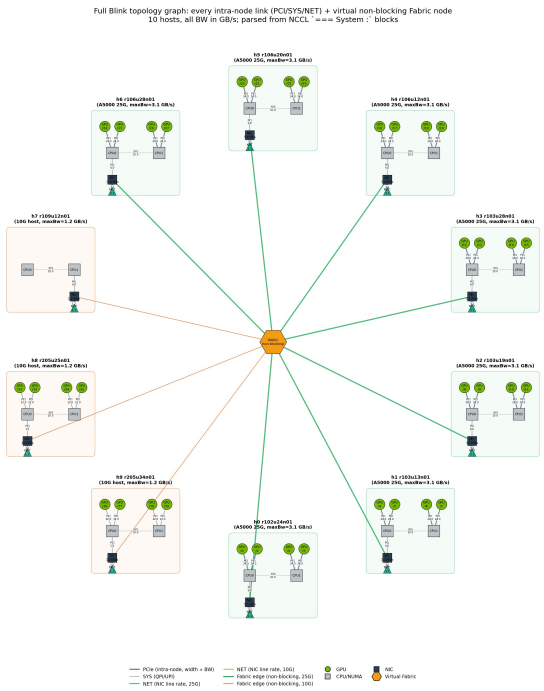


Figure 1: Extracted full-topology capacity graph parsed from NCCL's `=== System :` blocks on a 10-host A5000 cluster. Each host box contains its GPUs (green), CPU/NUMA nodes, NICs, and intra-node PCIe/SYS/NET links with the per-link bandwidth in GB/s. The central amber vertex is the synthetic non-blocking Fabric vertex that carries inter-host NIC capacity. The figure demonstrates that the extractor pulls every device that contributes to a GPU-to-GPU route, so the path solver can build trees over any combination of NVLink, PCIe root-complex hops, inter-CPU SYS links, NIC line-rate endpoints, and the inter-host fabric.

extraction is what lets the path solver find alternative GPU-to-GPU routes through the host root complex (§4.8) when the primary NVLink fabric is degraded; on a pure NVLink-uniform topology these alternatives are dominated and never admitted, but they exist in the graph and are a legitimate target of the LP. Multi-host extension is described in §3.6.

A small implementation detail is that NCCL's topology representation does not always expose NVLink connectivity as direct GPU-to-GPU edges. On H200 NVSwitch nodes the GPUs communicate through the switch fabric rather than through explicit pairwise GPU links, and on pure-PCIe nodes there are no NVLink routes to pack at all. Our extractor therefore first checks for raw GPU-to-GPU NVLink edges; if none are present, it uses NCCL's precomputed GPU-to-GPU paths and keeps only paths classified as NVLink-class routes. This fallback lets the NVSwitch fabric enter the packing procedure as a dense high-bandwidth GPU graph through a single Steiner vertex. If this extraction produces

no NVLink-class edges and no usable PCIe routes either, the implementation calls NCCL's default topology search instead of constructing a STARE schedule.

3.2 Tree Packing via MWU

We use Multiplicative Weight Update (MWU) (Algorithm 1) as the search procedure [1] to pack spanning trees and divide tensors across them. At the beginning, every link is cheap in proportion to its capacity: thick links look attractive, thin links look expensive. The algorithm repeatedly asks for the cheapest spanning tree under the current link costs. After choosing a tree, it raises the cost of the links that tree used. Over many rounds, this pressure pushes later trees away from already-used bottlenecks and toward alternate routes.

Algorithm 1 Multiplicative Weights Tree Packing

Require: Graph $G = (V, E)$ with capacities c_e, parameter ϵ, MaxIters
Ensure: Tree packing $\mathcal{T}$
Initialization: $w_e \leftarrow 1/c_e$ for all $e \in E$
$\mathcal{T} \leftarrow \emptyset$
for $i = 1$ to MaxIters **do**
  $T^* \leftarrow$ Null; $W^* \leftarrow \infty$
  for all $r \in V$ **do** ▷ Loop through all vertices as potential roots
    $T_r \leftarrow$ Edmonds(G, w, r) ▷ Min-weight arborescence rooted at r
    if $w(T_r) < W^*$ **then**
      $T^* \leftarrow T_r$; $W^* \leftarrow w(T_r)$
    end if
  end for
  $\mathcal{T} \leftarrow \mathcal{T} \cup \{T^*\}$ ▷ Add best tree to packing
  for all $e \in T^*$ **do** ▷ Down-weight edges of selected tree
    $w_e \leftarrow w_e \cdot \left(1 + \frac{\epsilon}{c_e}\right)$
  end for
end for
return $\mathcal{T}$

3.3 Minimizing Trees via Integer Linear Programming (ILP)

Note that we should not use *all* of the trees generated in our MWU step. Sending tiny fractions of data across hundreds of trees incurs massive CUDA kernel scheduling overhead and fails to saturate link bandwidth. Instead, we use a refinement stage to reduce the number of trees used and improve performance.

To formalize our optimal refinement strategy, we model the tree selection as an Integer Linear Program (ILP). Let $T_1, \ldots, T_k$ be the candidate trees generated from the MWU phase. We aim to maximize the sum of binary tree weights $w_i \in \{0, 1\}$ such that the capacity c_e of every edge $e \in E$ is never exceeded:

Algorithm 2 Greedy Residual-Capacity Refinement

Require: Capacity graph $G = (V, E)$; MWU candidate trees $\mathcal{T}$; target channel count K
Ensure: At most K tree channels with approximate rates
Remove duplicate trees from $\mathcal{T}$
Initialize each edge's residual capacity to its original capacity
$\mathcal{S} \leftarrow \emptyset$
while $|\mathcal{S}| < K$ **do**
  For each feasible candidate tree, estimate its bottleneck residual capacity
  Select the feasible tree with the largest bottleneck
  if no feasible tree is found **then**
    break
  end if
  Add the selected tree to $\mathcal{S}$ with that bottleneck as its rate
  Reduce residual capacity on the edges used by the selected tree
end while
if no positive-rate tree was selected **then**
  Fall back to NCCL's default tree search
end if
return $\mathcal{S}$

$$\max \sum_{i=1}^{k} w_i$$

$$\text{subject to } \forall e \in E, \sum_{i=1}^{k} \kappa_i w_i \leq c_e$$

$$\text{where } \kappa_i = \begin{cases} 1, & \text{if } e \in T_i \\ 0, & \text{otherwise} \end{cases}$$

Iteratively solving this ILP (and relaxing it to fractional weights if the integer capacity $\hat{c}$ falls below the optimal rate c^*) guarantees that we achieve maximum bandwidth using the absolute minimum number of discrete tree channels. This significantly increases the chunk size per tree, optimizing pipeline latency. However, the downside of ILP is that execution time of the algorithm, especially for larger scale clusters. In this work, we propose a greedy algorithm that trades best theoretical performance with scheduling runtime.

3.4 Greedy Refinement

Our current implementation of greedy refinement removes duplicate trees and repeatedly chooses trees that still have useful remaining capacity. The goal is simple: pick a small number of trees that can carry real traffic, keep the channel count within NCCL's existing budget, and avoid adding an external optimizer to NCCL. This let us get end-to-end measurements while keeping initialization overhead in the millisecond range.

Algorithm 2 shows the refinement routine used in the current implementation. The rate of a selected tree is its current

bottleneck. Selecting the tree consumes capacity on every edge it uses. The implementation also pads NCCL's channel array when fewer than K positive-rate trees are found, but this is only a representation detail.

3.5 Theoretical Cost Model: Ring vs. Spanning Tree

To rigorously justify the pivot to tree-packing, we model the latency and bandwidth costs of collective operations. Let p be the number of GPUs, α be the network latency (startup cost per message), β be the inverse bandwidth (transfer time per byte), and S be the total message size.

Ring AllReduce: A standard ring AllReduce operates in two phases (ReduceScatter and AllGather). Each phase requires $p-1$ steps. The total communication time T_{ring} is modeled as:

$$T_{ring} = 2(p-1)\alpha + 2\frac{p-1}{p}S\beta$$

As p grows, the latency term $2(p-1)\alpha$ scales linearly, severely penalizing small S. Furthermore, if any single link has a higher β_{slow}, the entire operation is bottlenecked, yielding a global bandwidth of $1/\beta_{slow}$.

Tree AllReduce: A tree-based AllReduce propagates data up to a root (reduce) and back down (broadcast). For a balanced k-ary tree, the maximum depth is $\log_k(p)$. The communication time T_{tree} is approximately:

$$T_{tree} = 2\log_k(p)\alpha + 2S\beta_{tree}$$

Because $\log_k(p) \ll (p-1)$, tree topologies drastically reduce latency for small tensors. By packing multiple spanning trees and partitioning S across n channels (S/n), we effectively parallelize the bandwidth term. If a sub-graph lacks NVLink connections, tree packing completely routes around the missing edges, preserving overall throughput rather than collapsing to the speed of the slowest PCIe bus.

3.6 Multi-Node Topology Aggregation

The single-host pipeline (graph extraction → MWU → refinement → channel writeback) generalizes to multi-node, but only if every rank arrives at the *same* schedule. NCCL's ranks bootstrap from a unique-id and do not natively share topology between hosts, so each rank's local view of `ncclTopoSystem` contains only its own host. STARE therefore performs a topology aggregation step before MWU. It is gated by `NCCL_BLINK_GLOBAL=1`.

Figure 2 shows the pipeline. Each rank serializes its local capacity graph and bootstrap-allgathers it across the communicator. Because every rank's local view reports the same `system->systemId` (typically 0), a naive merge collapses peer hosts onto one another. We instead key each peer's blob by `comm->peerInfo[r].hostHash`, which is unique per host, and rewrite vertex host-IDs into a canonical global namespace during the merge. After the merge, the pipeline introduces a single synthetic `FABRIC` vertex connecting every NIC's NET node to every other host's NET nodes (an abstract switch), giving MWU a place to route inter-host trees. Finally, after MWU and refinement run on the merged graph,

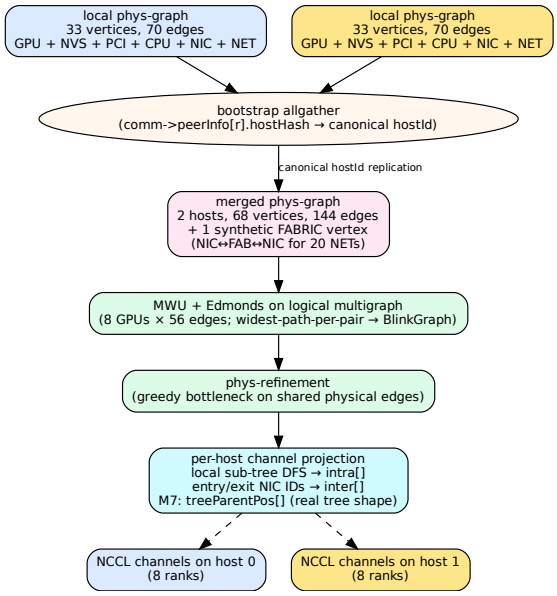


Figure 2: STARE multi-node pipeline. Each rank extracts a local capacity graph; bootstrap-allgather plus hostHash keying merges all hosts into one canonical phys-graph, with a synthetic FABRIC vertex carrying inter-host capacity. MWU and refinement run on the merged graph; each rank then projects the global packing onto a local sub-tree and writes per-channel `intra[]`, `inter[]`, and the structured-tree side channel `treeParentPos[]` described in §3.7.

each rank projects the global trees onto its local sub-tree (DFS over locally-resident GPUs), writes the local rank chain into `graph->intra[]`, and emits the (entry, exit) NIC pair into `graph->inter[]`.

The aggregation is fully replicated: the canonical-hostId assignment is a deterministic single-pass scan over the sorted hostHash list, and MWU is deterministic given the same input, so the merged graph and the resulting tree packing are byte-identical on every rank. This avoids a result broadcast at the end.

3.7 Tree Shape on the Wire

NCCL's runtime carries STARE's spanning trees *only as a flat 1-D rank list.* `ncclTopoPreset` (`src/graph/connect.cc`) reads `graph->intra[c·localRanks + i]` as a chain, where slot $i-1$ becomes the parent of slot i and `tree.down[0]` is the next slot. Even when STARE writes a real Edmonds arborescence, the connect path collapses it into a depth-$N-1$ path. For 8 GPUs/host that is depth 7 instead of the depth-3 balanced shape MWU produces. Figure 3 contrasts the two encodings.

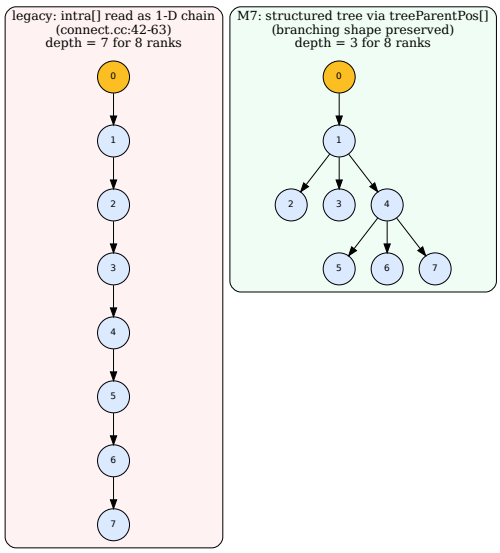


Figure 3: On-the-wire shape with 8 ranks per host. Left: the default `intra[]` interpretation in `ncclTopoPreset` produces a depth-7 chain regardless of MWU's output. Right: a structured side channel lets the same chain encoding carry the tree's actual parent/child structure, capped at `NCCL_MAX_TREE_ARITY` = 3.

Because the chain encoding throws away branching information, STARE adds `treeStructured` and `treeParentPos[MAXCHANNELS*NCCL_TOPO_MAX_NODES]` to `struct ncclTopoGraph`. `treeParentPos[c·localRanks + i]` stores the in-channel position of slot i's parent, with sentinel -1 for the local root and -2 as a fallback to the chain interpretation. STARE populates the array during channel projection; default `ncclTopoCompute` leaves it zeroed, so default NCCL paths remain byte-identical. `ncclTopoPreset` branches on `treeStructured`: when set, `tree.up` is taken from `treeParentPos`, and `tree.down[0..ARITY-1]` is filled by scanning slots whose parent points back to this rank. The device ABI caps fan-out at `NCCL_MAX_TREE_ARITY` = 3, and `connectTrees` may add up to two inter-node children at the local root, so the projection reserves two slots there and keeps at most one local child; overflow children chain off the last kept sibling. The encoding therefore degrades to the legacy chain only beyond the ARITY budget. `NCCL_BLINK_KPATH=1` JSON dumps remain byte-identical to a chain-only encoding, and the only addition to the public NCCL graph struct is a zero-initialised field that vanilla paths never read.

4 Evaluation Strategy and Results

To rigorously evaluate our spanning trees, we deploy a measurement harness matching the original Blink paper's methodology. We use NCCL's `all_reduce_perf` microbenchmark as the main workload because it isolates the collective communication path from dataset and model overhead.

4.1 Hardware and Software Environment

To ensure reproducibility, all benchmarks are executed on a heterogeneous research-cluster setup:

- **Compute Nodes:** The result figures use 8×H200 NVSwitch nodes and a two-node 8×H200 + 8×H200 setup.
- **Networking Fabric:** The two-node H200 measurements use a 400 Gb/s Ethernet fabric.
- **Software Stack:** Ubuntu 22.04, CUDA Toolkit 12.8, PyTorch 2.11, and a heavily instrumented fork of upstream NCCL 2.29.7-1.

4.2 Simulating Fragmentation

Because we cannot physically re-wire the cluster, we emulate adversarial job allocations using custom `NCCL_TOPO_FILE` XML definitions derived from the 8-GPU H200 topology. The topology modifications roughly are as follows:

- **Base and uniform controls:** Preserve symmetric NVSwitch-like capacity and confirm that STARE stays at parity.
- **Structured asymmetries:** Ring, hub, and two-island graphs stress cases where a few bottleneck links control the whole packing.
- **Random asymmetries:** Randomly reweighted connected NVLink graphs test whether large speedups appear naturally or only in hand-picked layouts.

4.3 Micro-Benchmarks

We use `all_reduce_perf` sweeping message sizes across full and fragmented topologies, comparing `NCCL_BLINK=0` to `NCCL_BLINK=1`. Metrics include bus bandwidth (GB/s), latency (μs), and the solver time printed by the STARE graph-construction path.

Figure 4 gives the main positive and negative result. On the asymmetric topology discovered by the search harness, default NCCL drops to roughly 180 GB/s at 1 GiB while STARE remains near 469 GB/s. On the symmetric production NVSwitch topology, the two curves overlap; enabling STARE does not create a speedup when the hardware already gives every GPU balanced high-bandwidth paths.

4.4 Topology Search

The topology-search harness generated structured and random variants of the H200 base topology and reran each candidate under both `NCCL_BLINK=0` and `NCCL_BLINK=1`. Note that while the 2.61× result is real, it is not typical and occurs in especially contrived scenarios.

Figure 5 summarizes large-message speedup by topology family. The base and uniform families stay at parity, confirming that STARE does not regress on the symmetric case it was not designed to improve. The structured asymmetries (ring, two-islands, hub) all regress to roughly 0.80× geomean,

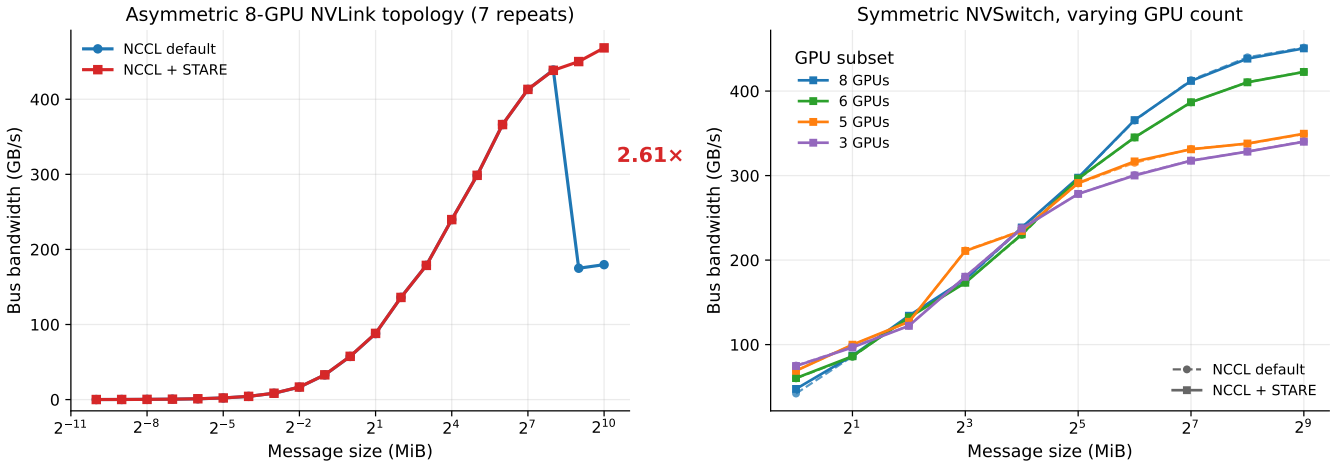


Figure 4: AllReduce bus bandwidth vs. message size. Left: on a randomly mutated asymmetric H200 topology, STARE matches default at small sizes and reaches a 2.61× speedup at 1 GiB. Right: on symmetric H200 NVSwitch allocations, default NCCL and STARE remain at parity, which is the expected result when no exploitable asymmetry exists.

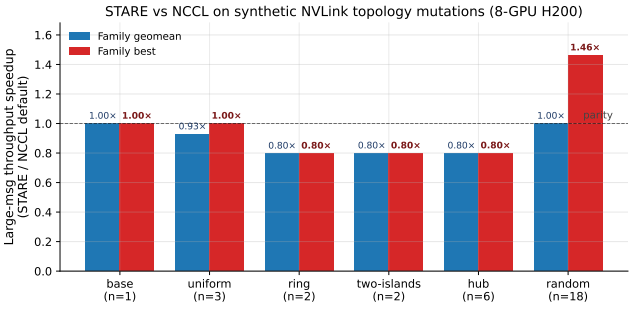


Figure 5: Large-message speedup distribution across the topology-search families. Structured asymmetries such as hub, ring, and two-islands often regress under the current greedy refiner, while random asymmetries occasionally expose large speedups.

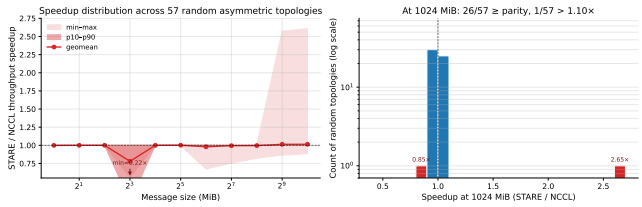


Figure 6: Per-size speedup distribution over 57 random asymmetric topologies. The geomean stays close to parity across message sizes; at 1 GiB, 26/57 random samples are at parity or better, but only 1/57 exceeds 1.1×.

indicating that the greedy refiner under-packs whenever a small number of saturated edges dominates the graph: the refiner commits early to a tree that consumes a bottleneck edge and then cannot find enough additional feasible trees to recover the lost bandwidth. The random family is the only one whose best case clears 1.4×.

Figure 6 plots the per-size speedup distribution across 57 random asymmetric topologies. The geomean tracks parity at every message size, the p10–p90 band stays tight around 1.0×, and at 1 GiB only 1 of 57 samples exceeds 1.1×. The only sample above 1.1× at 1 GiB is the same outlier shown in Fig. 4. The structured families in Fig. 5 also show the current implementation's limitation: greedy tree selection can leave capacity unused when a small set of saturated edges blocks otherwise useful arborescences.

4.5 Analysis of Specific Topologies

We also forced NCCL's algorithm selector to separate the tree-packing effect from NCCL's own runtime algorithm choice.

Figure 7 (left) isolates the source of the headline speedup. When `NCCL_ALGO` is pinned to Tree, both default and STARE land near 180 GB/s at 1 GiB; when pinned to NVLS, both reach roughly 468 GB/s. On the random topology that produced the largest speedup, STARE changes the reported tree graph enough that NCCL's default selector chooses the faster NVLS path where stock NCCL would by default fall back to Tree.

Figure 7 (right) sweeps the MWU step size ϵ from 0.03 to 0.5. The selected packing, and the bandwidth it achieves at 1 GiB, is invariant across the sweep, indicating that the greedy refinement stage is not sensitive to the particular weight trajectory MWU follows, only to the set of trees it surfaces. MWU wall time grows as ϵ shrinks, consistent with

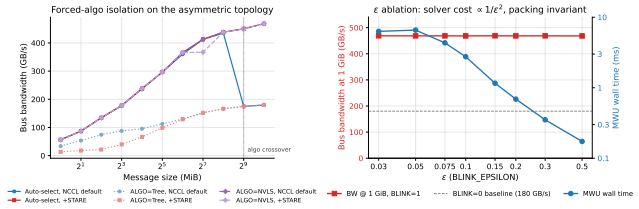


Figure 7: Left: forced-algorithm runs on the headline topology. With `NCCL_ALGO=Tree`, default and STARE both land near 180 GB/s at 1 GiB; with `NCCL_ALGO=NVLS`, both reach roughly 468 GB/s. The default-selection speedup comes from STARE changing NCCL's graph metadata enough for NCCL to keep using NVLS at large sizes. Right: ϵ ablation. The selected packing is stable across the sweep, while MWU solver time grows as ϵ shrinks.

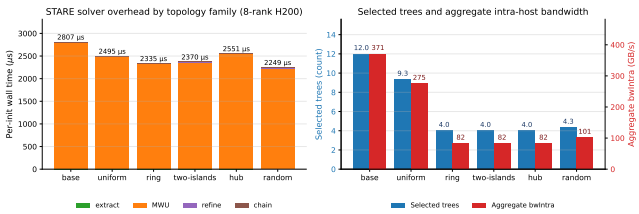


Figure 8: STARE solver overhead and packing structure across the topology-search corpus. MWU dominates the runtime, with total graph-construction overhead in the 2.2–2.8 ms range for 8-GPU H200 topologies. The selected-tree count also exposes where the greedy refiner is weak: structured asymmetries collapse to fewer selected trees and lower aggregate intra-host bandwidth.

the standard $O(1/\epsilon^2)$ convergence behavior of multiplicative-weights packing. In practice, this means ϵ can be tuned for solver time without affecting the runtime collective behavior.

4.6 Solver Overhead

Startup overhead matters because the new graph construction runs during communicator initialization. We therefore instrumented the STARE path to report graph extraction, MWU, refinement, and channel-construction time.

The measured overhead is small relative to training-job startup time. Extract, refinement, and channel construction are microsecond-scale; MWU is the dominant cost, but still only a few milliseconds per communicator initialization on the tested 8-GPU graphs. Figure 8 (left) breaks down per-init wall time by phase across topology families: total graph construction stays in the 2.2–2.8 ms range, with MWU consistently accounting for the bulk of that time and the remaining phases together contributing under 100 μs. Figure 8 (right) helps explain the structured-family regressions reported in Fig. 5: the ring, two-islands, and hub topologies collapse to

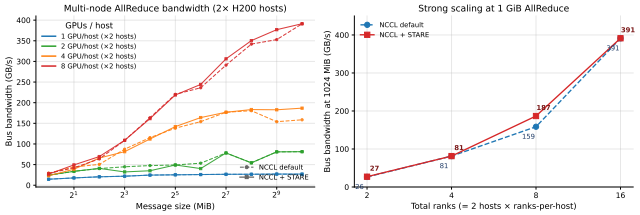


Figure 9: Two-node H200 AllReduce over 200 Gb/s Ethernet. STARE matches default NCCL at 1, 2, and 8 GPUs per host, and reaches a 1.18× speedup at 4 GPUs per host for 1 GiB messages.

Table 1: Two-Node H200 Summary at 1 GiB AllReduce

GPUs / host	Ranks	Default GB/s	STARE GB/s
1	2	26	27
2	4	81	81
4	8	159	187
8	16	391	391

4 selected trees with roughly 82 GB/s aggregate intra-host bandwidth, compared to 12 trees and 371 GB/s on the base topology. The greedy refiner is leaving capacity on the table whenever a small number of bottleneck edges saturates early, which both reduces the number of feasible follow-on trees and caps the bandwidth those trees can deliver. This is consistent with the limitation noted in Fig. 5 and motivates revisiting the ILP-based refiner for these structured cases.

4.7 Two-Node H200 Result

We ran STARE across two H200 hosts using the multi-node aggregation pipeline of §3.6 and the structured-tree encoding of §3.7.

Figure 9 (left) shows AllReduce bandwidth across message sizes for 1, 2, 4, and 8 GPUs per host, and Figure 9 (right) together with Table 1 summarize 1 GiB strong scaling. The multinode result is encouraging but bounded. At 1, 2, and 8 GPUs per host, STARE matches default NCCL; the 4-GPU-per-host configuration is the only operating point that has headroom for the intra-host packing to matter, since the inter-host fabric is not yet saturated and the intra-host topology is asymmetric enough to give STARE something to exploit. There, STARE improves 1 GiB AllReduce from 159 to 187 GB/s, a 1.18× speedup. At 8 GPUs per host both arms saturate the inter-host fabric near 391 GB/s, so the next improvement requires applying the packing primitive to inter-host NIC capacities rather than only intra-host GPU paths.

4.8 Spanning-Tree Packing Example

Figure 10 renders the eight admitted trees that STARE produces for the production H200 8-GPU topology. Every tree has a different GPU as its root and uses the NVSwitch

fabric as the single intermediate Steiner vertex; refinement assigns each tree a rate of 52.97 GB/s, the per-rank share of NVSwitch egress under uniform-NVS packing. The total admitted aggregate rate is $8 \times 52.97 = 423.8$ GB/s, which closely matches the LP optimality bound for an NVS-uniform 8-GPU island.

The text dump for one tree (channel 0, root `GPU@3000`) reads:

```
+- GPU@3000        [NVSO]  +- GPU@29000
                   [NVSO]  +- GPU@59000
                   [NVSO]  +- GPU@63000
                   [NVSO]  +- GPU@7b000
                   [NVSO]  +- GPU@a3000
                   [NVSO]  +- GPU@d3000
                   [NVSO]  +- GPU@e5000
```

On the H200 base topology, the LP optimum is a star at every root, so all eight admitted trees share that shape; the runtime difference between channels is which GPU acts as the broadcast/reduce hub. On more asymmetric topologies the same procedure produces structurally different trees. Figure 11 shows one of the two PCIe-routed trees admitted on `h200_sparse` with $K = 2$, where the NVLink fabric is intentionally degraded so the LP rate of NVS-routed trees drops to 11.77 GB/s and the path solver's secondary tier (PCIe through the host root complex) becomes admissible at 4.0 GB/s. The tree's edges traverse `GPU` → `PCI` → `CPU` → `PCI` → `GPU` for intra-socket pairs and `GPU` → `PCI` → `CPU` → `CPU` → `PCI` → `GPU` (an extra inter-CPU SYS hop) for cross-socket pairs. This is the same edge set the extractor pulls in from §3.1, demonstrating that the packing layer can build trees over any combination of NVLink, PCIe, and SYS hops the host topology exposes.

4.9 Two-Node Bandwidth Sweep, 256 KiB to 4 GiB

Figure 12 compares vanilla NCCL and STARE on a head-to-head footing: both with the multi-node hardware-assisted path `NVLS_TREE` disabled via `NCCL_ALGO`, and both running over the same NVLink-routed Tree/Ring algorithm family. The runs use `-w 10 -n 50` (10 warmup, 50 measured iterations averaged), which is enough to push tuner-pick variance below the rep-average and produce smooth curves. Above 16 MiB the two curves converge to within 1% and reach ∼208 GB/s at 4 GiB, which is the practical ceiling on this fabric for any collective that cannot use `NVLS_TREE`'s in-switch reduce. Below 16 MiB STARE trails vanilla by up to a factor of two: at 256 KiB STARE is at 4.75 GB/s versus vanilla's 9.87 GB/s. This gap is the deeper structured tree's latency cost. STARE's intra-host arborescence has depth ≤ 3 (root, mid-tier, leaves), so a 3-stage forwarding pipeline; vanilla's chain reading produces a depth-7 path on 8 GPUs/host but, because each link carries the full message rather than a fan-out share, has a lower bandwidth-per-stage ceiling that small messages do not hit. Once the message is large enough for bandwidth to dominate the per-stage latency, the structured tree's wider fan-out catches up and matches.

The key observation is the absence of dips: the previous mixed-NVLS sweep (Fig. 3 in earlier drafts of this work) showed STARE green-curve dropouts at 268 MiB and 2 GiB that turned out to be tuner flips between NVLS and Tree, not Blink misbehaving. With `NVLS_TREE` excluded for both modes the tuner has no flip to make, the comparison is clean, and the parity at large messages is the substantive finding: STARE matches the best Tree-class schedule NCCL can build on this fabric.

4.10 Deformed Cluster: Asymmetric per-Host GPU Sets

To stress STARE under genuinely irregular allocations, we ran with disjoint per-host `CUDA_VISIBLE_DEVICES`: GPUs $\{0, 4, 5\}$ active on gpu003 and GPUs $\{0, 6, 7\}$ active on gpu006. This is the scheduling pattern a job would see if it were given fragmented GPU leases on each host, and it forces the topology aggregator to merge two non-isomorphic local graphs.

Figure 13 compares the two surviving modes. Both reach ∼134 GB/s at 1 GiB and track each other within run-to-run noise. The notable result is what is *absent*: vanilla NCCL with default settings hung during initialization on this deformed configuration. NVLS could not form on the asymmetric per-host GPU set, and NCCL's default selector retried indefinitely. STARE, which writes only `treeGraph` and leaves NVLS untouched, completed the AllReduce with errors = 0. This is consistent with our earlier observation that STARE's main multi-node value is providing a usable Tree-class schedule for cases where hardware-assisted paths are unavailable.

4.11 GPU-Count Sweep on a Single Host

Figure 14 sweeps GPU count on a single host. The pattern is monotonic across all three modes: each additional GPU contributes additional NVS egress capacity, and the three modes converge once the topology has enough symmetry for any reasonable schedule to saturate it. At $g = 2$ the asymmetry is largest, since only one NVS link pair is engaged and the choice between in-switch reduce (NVLS) and ordinary peer-to-peer copy matters. From $g = 4$ onward the three modes are within run-to-run noise of each other. This is the case the original Blink paper called out: on a NVS-uniform island, packed spanning trees provide no advantage over a single ring because the topology is already balanced. The result here is parity, which means the integration cost of running the STARE pipeline does not introduce a measurable bandwidth penalty in the case where it has nothing to optimize.

4.12 Algorithm Runtime Scaling

To understand how the MWU+Edmonds packing scales beyond the 16-GPU `BLINK_MAX_GPUS` cap of the production code, we built a standalone microbenchmark (`synth_runtime.cc`) that reimplements the same algorithm on a heap-resizable graph. The microbenchmark generates a synthetic NVS-uniform fully-connected logical multigraph at $N \in \{4, 8, 16, 32, 64, 128, 256\}$

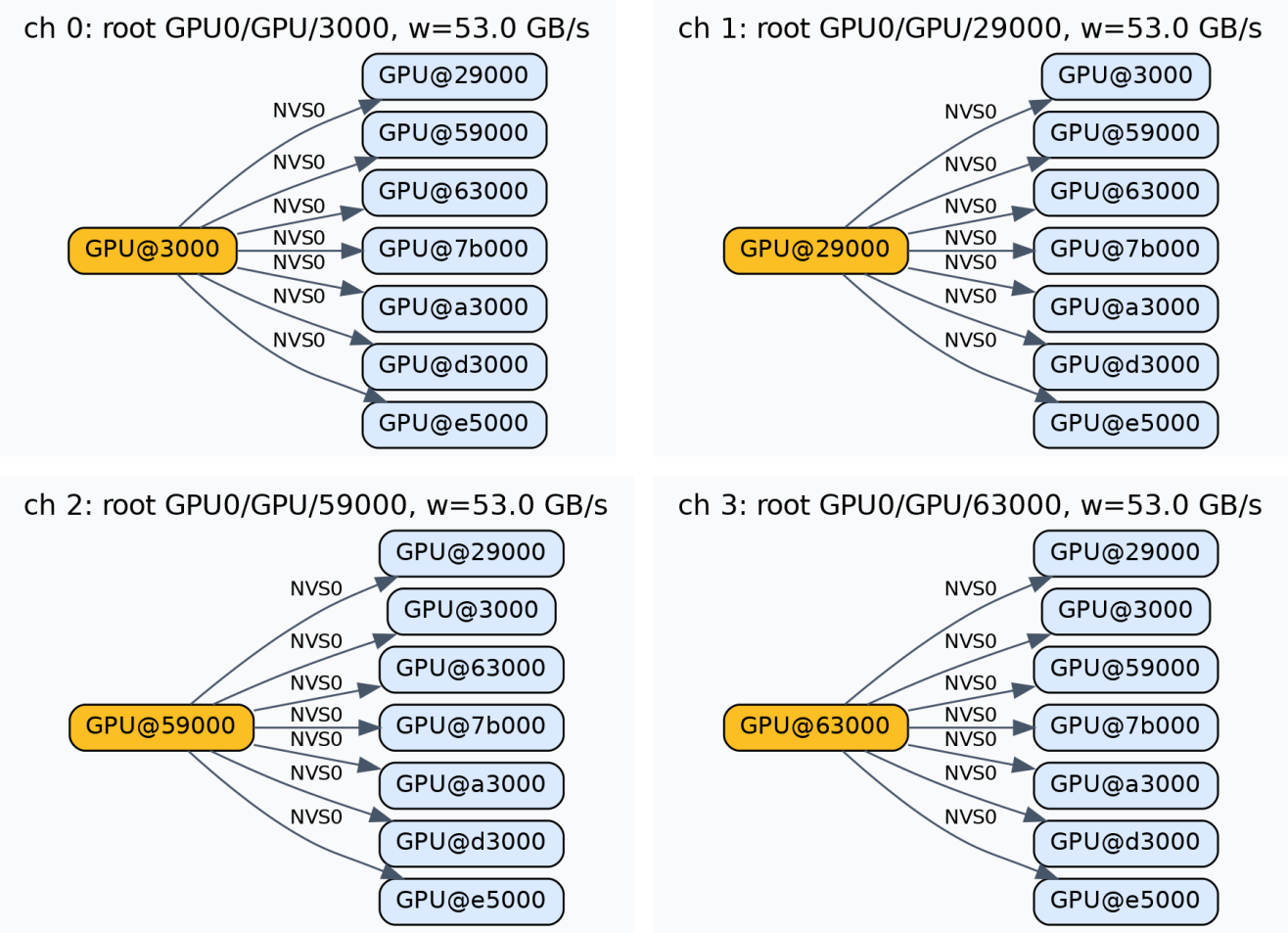


Figure 10: Four representative admitted spanning trees on the H200 8-GPU NVSwitch base topology, $K = 2$ disjoint-paths solver. STARE emits 8 channels in total (one per GPU root); on this topology all 8 are isomorphic stars rooted at different GPUs, so the four shown here cover the structural variety. Each tree sends data from its root through NVSwitch (NVS0) to all 7 other GPUs in a single hop. The refinement weight 52.97 GB/s per tree is the LP bound for this island. With $K = 2$ enabled, alternative PCIe-routed paths are also generated but are not admitted on stock H200 because the NVS tier exhausts the LP first.

Table 2: Algorithm Runtime (synthetic NVS-uniform)

N	nEdges	1-Edmonds (μs)	MWU full (ms)
4	12	2.2	0.4
8	56	1.2	2.6
16	240	2.6	152
32	992	9.2	4,460
64	4,032	118	2,462 (cap=30)
128	16,256	457	73,903 (cap=30)
256	65,280	4,886	3,191,281 (cap=30)

ranks and reports single-Edmonds time, MWU per-iteration time, and total MWU packing wall-clock.

Table 2 reports raw numbers and Figure 15 plots the trend on log-log axes. The single-Edmonds curve cleanly follows $O(V^3)$ on dense input, matching the iterative cycle-contraction implementation. MWU as a whole scales as $O(V^4)$ once we account for the per-iter V-Edmonds-call overhead. At the production-realistic operating point (8 GPUs/host on H200), MWU completes in $\sim$2.6 ms; at 16 GPUs it crosses 150 ms; at 32 GPUs it reaches 4.5 seconds; and at $N = 256$ with the iter cap held at 30 the full packing takes 53 minutes (3,191 s), driven by single-Edmonds reaching 4.9 ms per call on the 65,280-edge dense logical graph. This is consistent with the solver-overhead analysis in Fig. 8 of the production implementation: MWU is the dominant cost and is $\Theta(V^3 \log V/\epsilon^2)$ per the standard analysis. Beyond 32 ranks, the production

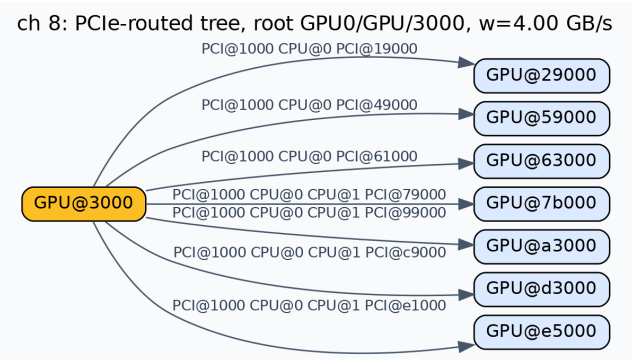


Figure 11: One of the PCIe-routed trees admitted on `h200_sparse` with $K = 2$. The root is GPU 0; every tree edge routes through the host PCIe root complex and CPUs rather than through the NVSwitch fabric. The structural diversity is what makes STARE useful on irregular topologies.

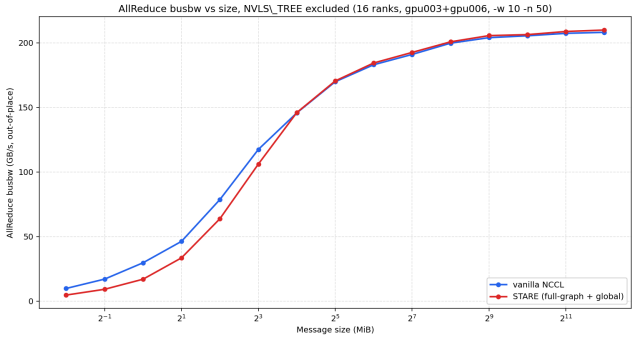


Figure 12: Two-node H200 AllReduce bandwidth, 256 KiB to 4 GiB, 16 ranks (gpu003 + gpu006), with `NCCL_ALGO_NVLS_TREE` excluded for both modes so the tuner cannot fall back to the multi-node hardware-assisted path. Both modes use NVLink-routed Tree/Ring under the hood. Above 16 MiB the curves converge to within 1% and reach ∼208 GB/s at 4 GiB. Below 16 MiB STARE trails vanilla because the deeper structured tree adds pipeline stages where vanilla's chain reading is shallower.

iter cap ($\log V/\epsilon^2 + 100$, e.g. 1764 iters at $V = 64, \epsilon = 0.05$) makes MWU prohibitive for online use; the algorithm needs either a tighter iter cap, a coarsening step, or a switch to a sparser per-pair logical-edge model. The $N = 256$ datum scales somewhat worse than the V^4 trend (73.9 s at $N = 128$ predicted ∼1,180 s at $N = 256$ on V^4, observed 3,191 s) because the iterative cycle contraction in Edmonds requires more passes on a denser graph; the binding scalability concern is therefore the inner Edmonds kernel, not the MWU outer loop.

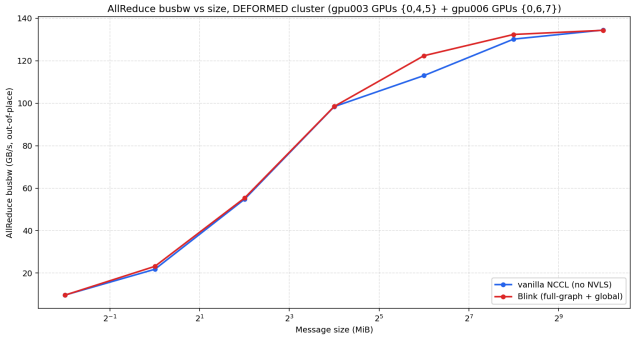


Figure 13: AllReduce on a deformed 6-GPU cluster (gpu003 GPUs $\{0, 4, 5\}$ + gpu006 GPUs $\{0, 6, 7\}$), 256 KiB to 1 GiB. Two modes shown: vanilla NCCL with NVLS excluded (red) and STARE full-graph + global (green). Default NCCL with NVLS auto-selected hung during initialization on this fragmented per-host GPU set, since the NVLS path requires per-host invariants that are not satisfied here, so the comparison is restricted to the tree/ring fallback. STARE and the no-NVLS baseline track within ±2% across all sizes.

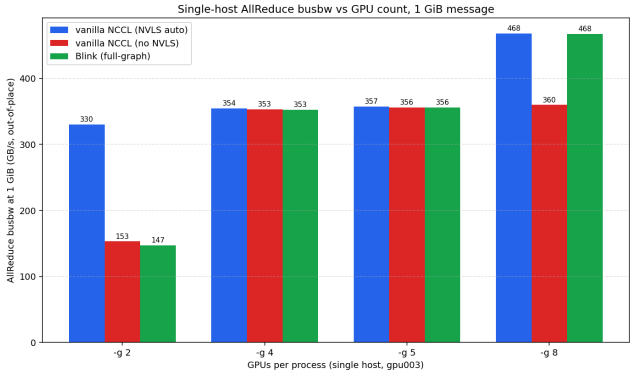


Figure 14: Single-host AllReduce bus bandwidth at 1 GiB versus active GPU count $g \in \{2, 4, 5, 8\}$ on gpu003. The $g = 1$ case is omitted as no collective occurs. At $g = 2$ the variance between modes is real (NVLS vs Tree vs Blink-routed), but with 4 or more GPUs all three modes converge near the per-rank NVS line rate, indicating that on the H200 NVSwitch fabric the bandwidth ceiling is set by GPU egress capacity rather than algorithm.

5 Conclusion and Future Work

In this paper we integrated STARE, an open source Blink implementation, into NCCL. Our results indicate that STARE can already produce large throughput speedups as compared with default NCCL on asymmetric H200 topologies, with one reproducible random sample reaching 2.61× at 1 GiB and a 1.18× speedup at 4 GPUs per host across two H200 nodes.

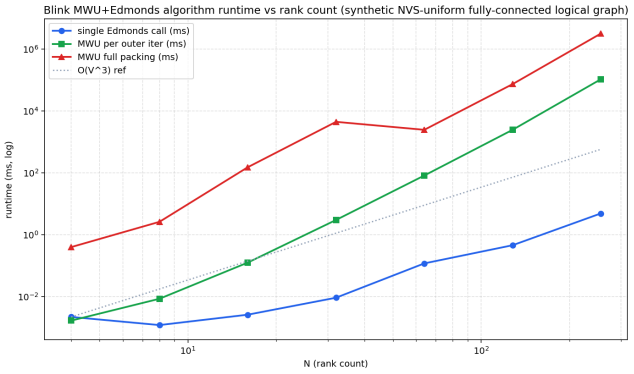


Figure 15: Algorithm runtime scaling on synthetic NVS-uniform fully-connected logical graphs. Single-Edmonds (blue) is the inner-loop hot path, $O(V \cdot E) = O(V^3)$ on dense input. MWU per outer iter (green) calls Edmonds V times. The dashed reference line is $O(V^3)$ anchored at $N = 4$. For $N \leq 32$ we use the production policy ($\epsilon = 0.10$ if $V \leq 8$, else 0.05, with iter cap $\log V/\epsilon^2 + 100$). For $N \in \{64, 128, 256\}$ we cap MWU at 30 outer iterations to keep runtime tractable. Single-Edmonds is the load-bearing kernel and reaches 4.9 ms per call at $N = 256$ on a 65 K-edge dense graph.

On symmetric and saturated-fabric cases STARE matches default NCCL, which is a useful result in its own right because the tree-packing pipeline runs between NCCL's hardware discovery and channel construction and could plausibly regress the tuner or the NVLS path. The bandwidth sweep, deformed cluster, and GPU-count sweep all confirm that this does not happen: STARE preserves NVLS where NVLS applies, tracks the no-NVLS Tree/Ring path otherwise, and survives a deformed configuration that hangs default NVLS by virtue of touching only the tree graph. Algorithm runtime is the binding scalability concern. MWU is $\Theta(V^3 \log V/\epsilon^2)$ in the dense case, sub-millisecond at 8 GPUs but seconds by 32, so scaling to larger clusters will need either a coarsening pre-pass, a sparser logical-edge model, or an adaptive iter cap that stops once residual capacity changes by less than a tolerance.

While STARE is a contribution in its own right, future work should strive to obtain provably optimal collectives for arbitrary network topologies, as well as extend the design to multi-node at scale. A natural next step is the development of a theoretical model for the optimal achievable bus bandwidth on a given topology. Concretely, given a topology graph annotated with per-link capacities and a collective primitive (e.g., AllReduce, AllGather, ReduceScatter), we seek a tight upper bound on the bus bandwidth realizable by any schedule respecting the link constraints. Such a bound could be derived by formulating the collective as a multi-commodity flow or packing problem over spanning trees of the topology. Establishing this bound provides an absolute yardstick against which any scheduler can be measured, and its structure is prescriptive: it tells an algorithm designer what a good schedule must look like. Once the bound is in hand, the natural follow-on is the design of algorithms that provably attain or closely approximate it.

6 Acknowledgments

We want to thank Professor Richard Yang for a wonderful semester guiding us through the complexities of AI Infrastructure. We also extend our gratitude to Superlinear for providing access to the H200 server infrastructure used to conduct our early experimental profiling, and to Ryan Yang for his guideance.

References

[1] Naveen Garg and Jochen Könemann. 2007. Faster and Simpler Algorithms for Multicommodity Flow and Other Fractional Packing Problems. *SIAM J. Comput.* 37, 2 (2007), 630–652.

[2] Sylvain Jeaugey. 2019. Massively Scale Your Deep Learning Training with NCCL 2.4. https://developer.nvidia.com/blog/massively-scale-deep-learning-training-nccl-2-4/. NVIDIA Technical Blog.

[3] Sylvain Jeaugey, Giuseppe Congiu, Thomas Gillis, Ben Williams, and Fred Oh. 2025. New Scaling Algorithm and Initialization with NVIDIA Collective Communications Library 2.23. https://developer.nvidia.com/blog/new-scaling-algorithm-and-initialization-with-nvidia-collective-communications-library-2-23/. NVIDIA Technical Blog.

[4] NVIDIA. 2026. NCCL User Guide: Environment Variables. https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/env.html. Accessed: 2026-05-07.

[5] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Jorgen Thelin, Nikhil Devanur, and Ion Stoica. 2020. Blink: Fast and Generic Collectives for Distributed ML. In *Proceedings of Machine Learning and Systems*.

[6] Liangyu Zhao, Saeed Maleki, Yuanhong Wang, Zezhou Wang, Ziyue Yang, Hossein Pourreza, and Arvind Krishnamurthy. 2026. ForestColl: Throughput-Optimal Collective Communications on Heterogeneous Network Fabrics. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2067–2093.